Non-Interactive Distributed Key Generation on the EVM with Per-Application Committee Keys

Anonymous Submission

Abstract. We present a distributed key generation protocol on a public EVM chain in which every contribution is a publicly verifiable Feldman dealing carried by a Groth16 proof and verified with one fixed-size check over eight public inputs, whatever the committee size, so no complaint phase exists. One epoch deals a pool of $K_{\max}$ independent committee keys inside the same proof, and a second proof, one per epoch, finalizes it: the contract stores all $K_{\max}$ keys and their share-commitment roots in one transaction. Every application claims its own pool key, so a copied ciphertext decrypts under an unrelated key, or adds an organizer key revealed once, application-wide, before which the contract admits no partial decryption. A secrecy game with corrupt organizers, submitters and f committee members admits DDH reductions for both modes under stated corruption, random-oracle and proof-system assumptions; the automatic mode additionally relies on Groth16's weak simulation-extractability in the algebraic group model. At a committee bound of 32 and a pool of 16, the contribution circuit has 5.90 M constraints and proves in 3.5 s; a contribution costs 0.50–1.31 M gas and the finalization of all 16 keys 2.2–2.8 M.

Keywords: Distributed key generation · Threshold decryption · Publicly verifiable secret sharing · zk-SNARKs · EVM.

1 Introduction

A distributed key generation protocol lets n parties produce a public key whose secret is Shamir-shared so that any t of them can decrypt [18], while nobody ever holds it. Pedersen-style DKGs [36] let recipients check their own shares and settle faulty dealings by interactive complaints; on a public chain that dispute logic runs in a contract, at block granularity, and is itself an attack surface [37,43]. Publicly verifiable secret sharing [44,38,22] removes complaints in principle, but its proofs cost $\Theta(n)$ group operations per dealing, in groups the EVM does not expose or prices at hundreds of thousands of gas.

A general-purpose succinct proof changes the accounting: a Feldman dealing carried by a Groth16 proof is accepted or rejected by one fixed-size verification with eight public inputs, whatever n and t; what grows with the committee is the calldata of commitments and encrypted shares, so a contribution transaction costs 0.50 M gas at $n = 4$ and 1.31 M at $n = 32$ in our build (§7). The rest of the paper makes that check bind the right data and lets one committee serve many applications under keys that do not leak into each other.

Motivating application. Consider an encrypted voting service: ballots are exponential-ElGamal ciphertexts, aggregated homomorphically off chain, and only authorized tally ciphertexts are submitted for threshold decryption. The aggregate submitter does not know the summed randomness, so the proof of knowledge with which threshold ElGamal classically closes its decryption oracle [42] is unavailable. Elections need isolated keys; some also need an organizer-controlled, application-wide release. Ballot validity, eligibility, aggregation correctness and tally authorization lie outside this layer; the protocol alone does not stop an authorized submitter from submitting an individual ballot.

Contributions.

1. *Succinctly verified dealings of a key pool.* Each contribution deals $K_{\max}$ independent Feldman polynomials to the whole committee and is verified on chain by one fixed-size Groth16 verification with eight public inputs; an invalid dealing is refused at submission, so no dispute phase exists. One finalization proof per epoch aggregates the accepted dealings of all $K_{\max}$ keys and evaluates every member's share commitments; the contract stores $K_{\max}$ keys and Merkle roots in one transaction, so every key is usable the moment the epoch is live (§4).
2. *Per-application keys in two modes, with a security model.* Every application claims one pool key. An *automatic* application is decrypted by any t members inside its decryption window; an *organizer-locked* one adds an organizer key, $\mathsf{PK}_{\mathtt{aid}} = P_j + \mathsf{PK}_{\mathtt{org}}$, whose secret is revealed once, application-wide, before which honest nodes (fewer than t corrupt) release no partial. A secrecy game with corrupt organizers, submitters and f members yields DDH reductions for a locked application before its reveal ($n - f \geq t$, Theorem 2) and for an automatic one ($f < t$, Theorem 1), with their assumptions stated (§6).
3. *Verification devices and an implementation.* A binding random-linear-combination commitment, gated by public counts so calldata carries no padding, keeps every verifier at 7–15 public inputs; we state its exact hypotheses and the forgery when one is dropped (Lemma 1). A point-based Vandermonde check pins the combine proof's interpolation vector (§5). Three contracts and four circuits are measured with their real verifiers on a local testbed (§7).

2 Related Work and Comparison

Pedersen's DKG [36] runs n Feldman instances [20] in parallel; Gennaro et al. showed that a rushing adversary biases its key [24] and that the biased key stays safe for discrete-log applications such as ElGamal [25], the position this paper takes. Kate and Goldberg moved DKG to the Internet setting [32]; asynchronous DKGs [34,17,3,16,47,2,21] reach near-quadratic communication and adaptive security but rest on asynchronous VSS and agreement primitives [4], unneeded on a chain that already orders messages; SPRINT [8] targets threshold signing.

Complaint-free dealing is not new. Publicly verifiable secret sharing [44,38] and Fouque and Stern's one-round DKG [22] began the line; SCRAPE [12] made

Table 1. Dealing verification in DKGs with a public bulletin board. PV: publicly verifiable dealing. SRS: the scheme needs a per-circuit structured reference string; unmarked rows need no trusted setup. Size: what one dealing publishes.

Protocol	Check	Verifier per dealing	Size per dealing	Verified by
Pedersen; Gennaro et al.; Shutter [36,24,45]	complaints	$O(t)$ exp., recipient only	$O(t) + n$ private	parties; synchronous rounds
Fouque–Stern; SCRAPE; ALBATROSS; Kate et al. [22,12,13,33]	PV	$O(n)$ Paillier, group or class-group ops	$O(n)$	anyone
Gurkan et al.; Groth NI-DKG [29,28]	PV	$O(n)$ exp. + pairings; aggregatable	$O(n)$	anyone; BFT replicas
Ferveo [6]	PV	$O(n)$ exp. + pairings	$O(n)$	BFT validators
EthDKG [37]	complaints on chain	$O(t)$ exp. per recipient; contract adjudicates	$O(t + n)$	EVM contract; block windows
Sober et al. [43]	complaints on chain	SNARK-checked rebuttal (SRS)	$O(t + n)$	EVM contract; block windows
This work	PV	one Groth16 verification, 8 public inputs (SRS)	$K_{\max}(2t+n)$ $+5n$ words, $K_{\max}$ dealings	EVM contract; block windows

verification linear, ALBATROSS [13] batched it, Gurkan et al. [29] made transcripts aggregatable, Groth's NI-DKG [28] uses forward-secure encryption and a bespoke proof system, Kate et al. [33] need no setup at all with class groups, and Ferveo [6] deploys publicly verifiable dealings among the validators of a BFT chain, each verifier performing $\Theta(n)$ group operations per dealing.

On the EVM, EthDKG [37] ports Joint-Feldman with on-chain dispute resolution, Sober et al. [43] check the rebuttal with a zk-SNARK but keep the complaint phase, and Shutter [45] runs Feldman VSS with interactive complaints in production. Table 1 positions the present design: a publicly verifiable dealing, checked by the contract at a fixed verifier cost, under a per-circuit trusted setup; it separates verifier work from transcript size.

3 Model, Goals and Notation

3.1 Notation and Primitives

$\mathcal{G}$ is a cyclic group of prime order q, written additively with generator G; DDH is assumed hard. The implementation uses BabyJubJub's prime-order subgroup (cofactor 8) over the BN254 scalar field $\mathbb{F}_p$, $p > q$, the circuits' native field, so in-circuit reduction modulo q needs a bounded quotient (§5). $\mathsf{H_K}$ is Keccak-256, reduced modulo q for the contract's Schnorr challenges and modulo p for the calldata-binding challenges of §5; $\mathsf{H_P}$ is Poseidon [26]. The contract checks points canonical, on-curve and, where meaningful, non-identity; whoever multiplies a secret into a point checks its subgroup. An epoch has $n \le N_{\max}$ members, threshold t, f corrupt, and $K_{\max}$ pool keys ($N_{\max} = 32$, $K_{\max} = 16$); $\mathsf{QUAL} \subseteq [n]$, $|\mathsf{QUAL}| \ge m_{\min} \ge t$, are the accepted dealers, frozen at finalization; $\mathcal{Q}$, $|\mathcal{Q}| = s \ge t$, those whose partials are combined.

Feldman VSS [40,20]. A dealer with secret σ samples a uniform f of degree below t over $\mathbb{F}_q$ with $f(0) = \sigma$, sends party u the share $f(u)$ and publishes $C(k) := a_k G$ per coefficient; u checks

$$f(u)\, G \stackrel{?}{=} \sum_{k=0}^{t-1} u^k\, C(k). \tag{1}$$

Hashed ElGamal for shares. A share $s \in \mathbb{F}_q$ is encrypted under a recipient key $\mathsf{pub} = xG$ as

$$(R, \sigma) := \big(rG,\ s + \mathsf{H_P}(\mathsf{ctx} \,\|\, r\,\mathsf{pub}) \bmod q\big), \qquad r \leftarrow \mathbb{F}_q, \tag{2}$$

and recovered as $s = \sigma - \mathsf{H_P}(\mathsf{ctx} \,\|\, xR)$; ctx binds epoch, dealer, recipient and key index, so one ECDH value per recipient yields $K_{\max}$ distinct masks; IND-CPA holds under hash-CDH [1,14]. Applications use exponential ElGamal (§4).

3.2 Chain, Parties and Adversary

Chain and parties. We assume finality below the parties' confirmation depth, bounded inclusion of funded honest parties' transactions (phase windows are sized accordingly) and availability of accepted calldata. Registered operators form each epoch's committee by snapshot-based eligibility filtering from a block-hash seed (§4), exposed to proposer bias and inclusion races: every result is conditioned on the selected committee meeting the corruption bound. Organizers register applications and submitters post ciphertexts; registry admission is a deployment assumption.

Adversary. Corruptions are static [10]: f members and any number of other operators, organizers and submitters. $n - f \geq t$ gives availability and lets the challenger of §6 interpolate every pool secret without rewinding; $f < t$ plus an honest accepted dealer ($m_{\min} > f$) is needed wherever a pool secret must stay hidden; honest members keep shares for the epoch's decryption horizon. We assume DDH in $\mathcal{G}$, $\mathsf{H_K}$ and the share-mask Poseidon as random oracles, Poseidon collision resistance, knowledge soundness and zero knowledge of the SNARK—Groth16 [27] needs a per-circuit structured reference string and is knowledge-sound only in the generic-group model, which every dependent statement inherits—and, for the automatic mode only, weak simulation-extractability of Groth16 in the algebraic group model [5]; §6 states these as (a)–(f) and which result uses which. Adaptive or proactive corruption, UC composition and setup compromise are out of scope.

3.3 Goals

Correctness: for a key j of a live epoch, $A_j := \sum_{\ell \in \mathsf{QUAL}} f_\ell^{(j)}$, the stored key is $A_j(0)\, G$, the Merkle leaves $A_j(i)\, G$, honest member i's share $A_j(i)$, and a combined plaintext of a well-formed ciphertext $\log_G(C_2 - \mathsf{sk_{aid}} C_1)$ within the

Table 2. Guarantees by adversary. Outsiders: other organizers and submitters. Gated: honest members post no partial before the application's window opens and, if locked, before its reveal. Row 1 also needs one honest accepted dealer.

Adversary controls	Bound	What holds
Outsiders; f members	$f < t$, $n - f \geq t$	Honest application confidential while gated (Theorem 1)
Own organizer (correctly generated secret, unrevealed); f members	$f < t$, $n - f \geq t$	No leak via other applications; own one gated (Theorem 2)
Outsiders; f members	$n - f \geq t$	Locked application confidential until reveal (Theorem 2)
t members	—	Automatic mode broken; locked keeps its organizer factor
Organizer and t members	—	No embargo in either mode; coalition decrypts off chain
Authorized submitter	any	Chooses what enters; nothing is claimed about its inputs

search bound. *Availability*: an epoch with $m_{\min}$ accepted contributions goes live, by one finalization proof, with every key usable; a well-formed ciphertext whose gate is open is decrypted whenever t honest holders are available and included. *Secrecy*: the game of §6; Table 2 states what holds under which corruption, the gate bounding only what the contract accepts and honest nodes publish, never t colluding share holders.

4 Protocol and Design Rationale

The protocol runs in *epochs*. An epoch `eid` deals a pool of $K_{\max}$ independent keys $P_0, \ldots, P_{K_{\max}-1}$; each application registered against it claims one key. Operators hold registered share-encryption keys pub_i (§7); admission is a deployment concern. On the block axis an epoch passes a selection window, a contribution window and a key-assembly gap, then stays LIVE with all $K_{\max}$ keys stored.

4.1 Committee Selection

The initiator submits $(t, n, m_{\min}, \alpha)$ (threshold, committee size, minimum accepted contributions, oversubscription) and the contract snapshots the number N of active operators, pins a seed block b_s a few blocks ahead and stores $\tau := \lfloor 2^{256} \alpha n / N \rfloor$. The first claim after b_s freezes $\mathsf{seed} := \mathsf{blockhash}(b_s)$; operator op is admissible iff $\mathsf{keccak}(\mathsf{seed} \,\|\, \mathsf{op}) < \tau$, and the first n admissible claimants take the slots; only operators active before creation may claim. This is snapshot-based *eligibility filtering*, not an unbiasable sampler: the seed's proposer can resample or withhold it, and inclusion latency decides among admissible claimants; §6 assumes the selected committee meets the corruption bound.

Epochs are created permissionlessly once EPOCH_DURATION blocks have elapsed, or earlier once the newest epoch is LIVE with at most one unclaimed key or aborted; deployment immutables floor t and n and cap α, which a creation-race winner can pin every epoch to. Robustness depends on the chosen epoch's $n - t$: the floors enforce no margin, since $t = n$ is admissible, so tolerating r unavailable members needs $t \leq n - r$.

4.2 Epoch Key Generation

Contribution. Each member i samples, for every key j, $0 \le j < K_{\max}$, a uniform polynomial $f_i^{(j)}(x) := \sum_{k<t} a_{i,j,k}\, x^k$ over $\mathbb{F}_q$ and one nonce $r_{i,u}$ per recipient u, and publishes in one transaction the commitments $C_i^{(j)}(k) := a_{i,j,k}\, G$ and, for every (j,u), the encrypted share

$$E_i^{(j)}(u) := \big(R_i(u), \sigma_i^{(j)}(u)\big) = \big(r_{i,u}G,\ f_i^{(j)}(u) + \mathsf{H_P}(\mathsf{ctx}_{i,u} \| j \| r_{i,u}\mathsf{pub}_u) \bmod q\big)$$

and a proof π_i that (1) holds and every $E_i^{(j)}(u)$ encrypts $f_i^{(j)}(u)$ (§5). The commitments are digested per key, and the $K_{\max}$ digests again into a single `commitmentsHash`, which the contract stores after verifying π_i; the shares stay in calldata, whose $K_{\max}(2t+n)+5n$ words are sized by the epoch's (t,n), not by the circuit's capacity (§5). By Lemma 2 an accepted contribution is a correct dealing to all n recipients, so no complaint phase [36,24] is needed.

Finalization. Let $A_j := \sum_{\ell \in \mathsf{QUAL}} f_\ell^{(j)}$, $\overline{C}^{(j)}(k) := \sum_{\ell \in \mathsf{QUAL}} C_\ell^{(j)}(k)$, $P_j := \overline{C}^{(j)}(0) = A_j(0)\, G$ and $D_{j,i} := \sum_{k<t} i^k\, \overline{C}^{(j)}(k) = A_j(i)\, G$ for $i \in [n]$. After the contribution window, with at least $m_{\min}$ accepted contributions and the key-assembly gap elapsed, anyone may call $\mathsf{finalizeEpoch}$, with no deadline, carrying one proof for the whole pool: that the rows of its transcript name distinct accepted dealers $\mathsf{QUAL} \subseteq [n]$ and reproduce their stored digests, that the $\overline{C}^{(j)}(k)$ aggregate their commitments for every key j, and that the $D_{j,i}$ are the evaluations at every committee position (§5). In one transaction the contract freezes QUAL, stores every P_j and $\mathsf{root_j}$, a Keccak Merkle root over $N_{\max}$ tagged leaves $\mathsf{keccak}(\texttt{0x00} \,\|\, D_{j,i})$, $i \le n$, a constant empty leaf beyond n and inner nodes $\mathsf{keccak}(\texttt{0x01} \,\|\, l \,\|\, r)$, and marks the epoch LIVE; there is no per-key activation step. An epoch short of its committee or its $m_{\min}$ rows is dead. Every member $i \in [n]$ obtains $e_{j,i} := A_j(i)$ by decrypting the calldata, whether or not it dealt: decryption tolerates $n - t$ absent members.

4.3 Applications and Their Keys

Registration against a LIVE epoch names a non-zero field element `aid` and a policy, and claims the next unclaimed key P_j; every key of a live epoch is usable, so it fails only once the pool is exhausted. In the *organizer-locked* mode the registrant, the organizer, also submits $\mathsf{PK_{org}} = \mathsf{sk_{org}}G$ with a Schnorr proof of possession $(A, z) = (wG,\ w + c\,\mathsf{sk_{org}})$, where $c := \mathsf{H_K}(\mathsf{dom_{org}} \| \texttt{eid} \| \texttt{aid} \| \mathsf{PK_{org}} \| A)$, checked as $zG = A + c\,\mathsf{PK_{org}}$; it excludes rogue keys such as $-P_j$. The application key is $\mathsf{PK_{aid}} := P_j + \mathsf{PK_{org}}$. In the *automatic* mode there is no organizer key: the contract records the identity and 0, $\mathsf{PK_{aid}} := P_j$, and any t share holders decrypt.

A user encrypts m as $(C_1, C_2) := (rG,\ mG + r\,\mathsf{PK_{aid}})$, $r \leftarrow \mathbb{F}_q$: exponential ElGamal, additively homomorphic, m being recovered from mG by a bounded search. Since every key is dealt by independent polynomials, the committee's term $A_j(0)\, C_1$ is useless under any other application's key (Theorems 1 and 2).

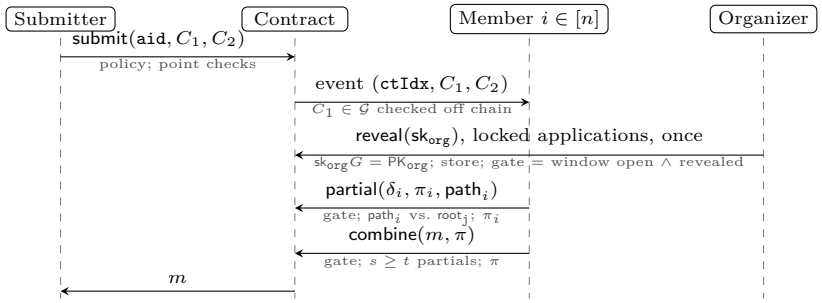

Fig. 1. Decryption on pool key P_j; arrows into the contract are transactions. Partials and the combine carry Groth16 proofs, partials also a Merkle path for $D_{j,i}$; gate: window open and, if locked, revealed.

Policy. Registration fixes who may submit (anyone, an allow-list or the registrant), a submission block window and a decryption window. Ciphertexts are accepted only inside the submission window and before decryption closes; partials and combines only inside the decryption window and, for a locked application, after the reveal. Honest nodes (fewer than t corrupt) release no partial before this gate opens; t colluding members are not bound (§8).

4.4 Threshold Decryption

Submission. submitCiphertext(aid, C_1, C_2) carries no proof (Figure 1): the contract enforces the policy, assigns ctIdx on chain and checks both points well formed and not the identity, leaving the prime-order check to the member about to multiply its share into C_1, since a cofactor component would leak $e_{j,i} \bmod 8$ through its partial.

Partial decryption. Any member $i \in [n]$ publishes $\delta_i := e_{j,i}C_1$ with a DLEQ proof [15] $(A_i, B_i, z_i) := (w_iG,\ w_iC_1,\ w_i + c_i\, e_{j,i})$, $w_i \leftarrow \mathbb{F}_q$, with challenge $c_i := \mathsf{H_P}(\mathsf{dom} \| \mathtt{eid} \| \mathtt{aid} \| \mathtt{ctIdx} \| i \| D_{j,i} \| C_1 \| \delta_i \| A_i \| B_i)$, verified as $z_iG = A_i + c_iD_{j,i}$ and $z_iC_1 = B_i + c_i\,\delta_i$ inside a Groth16 proof (§5), and a Merkle path for $D_{j,i}$ that the contract checks against $\mathsf{root_j}$ at leaf $i - 1$.

Reveal. A locked application's organizer publishes $\mathsf{sk_{org}}$ through the permissionless, one-shot revealOrganizerSecret call; the contract checks $\mathsf{sk_{org}}G = \mathsf{PK_{org}}$ and stores it. Until then no partial or combine of the application exists on chain; afterwards every past and future ciphertext is eligible, so the organizer decides *when*, not *which* (§8).

Combine. Once $s \geq t$ partials from $\mathcal{Q} = \{x_1, \ldots, x_s\}$ are on chain and the gate is open, anyone submits a proof establishing, with the canonical Lagrange

coefficients λ_k of $\mathcal{Q}$ (§5),

$$\sum_k \lambda_k \, \delta_{x_k} = A_j(0)\, C_1, \qquad M = C_2 - \sum_k \lambda_k \, \delta_{x_k} - \mathsf{sk}_{\mathtt{org}}\, C_1 = mG, \tag{3}$$

and, from the private witness $\mathsf{sk}_{\mathtt{org}}$, that $\mathsf{sk}_{\mathtt{org}}G$ is the $\mathsf{PK}_{\mathtt{org}}$ on record, trivially for an automatic application. The combiner recovers m off chain by baby-step giant-step [41] (§7).

Design rationale. Clients already produce exponential ElGamal ciphertexts and homomorphic aggregates, the organizer runs no prover, and the DKG verifies nothing application-specific. A proof of knowledge of r with $C_1 = rG$ [42] is unavailable, since an aggregate's submitter does not know $\sum_k r_k$; the committee therefore answers any C_1 under any key it serves, so separation cannot come from a public additive offset ($\mathsf{PK}_{\mathsf{ep}} + \mathcal{H}(\mathtt{aid})\, G$ [46] is removed by shifting C_2 by $\mathcal{H}(\mathtt{aid})\, C_1$) but only from independently dealt keys, pooled at one transaction and proof per member plus one finalization proof per epoch.

5 Succinct Verification

Recurring operations are proven in Groth16 circuits [27] over BN254 with BabyJubJub embedded and Poseidon [26] as hash. Circuits have fixed capacity $N_{\max} = 32$, $K_{\max} = 16$ (inactive slots masked to constants), check points on the curve, range-check scalars used modulo q and use hint-free variable-base multiplication [31]. Contribution $\mathcal{R}_{\mathsf{deal}}$ has public inputs $(\mathtt{eid}, t, n, i, h_C, h_E, \rho, \beta)$; finalization $\mathcal{R}_{\mathsf{fin}}$ has $(\mathtt{eid}, t, n, a, d, \rho, \beta)$, $a = |\mathsf{QUAL}|$, with the transcript digest d at index 4 of 7; partial decryption $\mathcal{R}_{\mathsf{part}}$ has the 15 scalars $(\mathtt{eid}, \mathtt{aid}, \mathtt{ctIdx}, i, C_1, D_{j,i}, \delta_i, A_i, B_i, z_i)$, c_i as in §4; combine $\mathcal{R}_{\mathsf{comb}}$ has $(\mathtt{eid}, \mathtt{aid}, \mathtt{ctIdx}, t, s, h, m, \rho, \beta)$, $s = |\mathcal{Q}|$. Indices range over keys $j \in \{0, \ldots, K_{\max} - 1\}$, $u, i \in [n]$, dealer rows $\ell < a$, $k < t$ ($k \le s$ in $\mathcal{R}_{\mathsf{comb}}$); all else is witness, $\mathbf{w}$ the transcript committed by $\beta = \mathsf{BRLC}_\rho(\mathbf{w})$ and $h_C \,\|\, h_E$, d, h its Poseidon digests (§5.1); $g_{\ell,j} := \mathsf{H_P}(\mathbf{C}_\ell^{(j)})$ digests one dealer's key-j commitments:

$$\begin{aligned}
\mathcal{R}_{\mathsf{deal}} :\ & a_{i,j,k} \in [0,q), \quad C_i^{(j)}(k) = a_{i,j,k}\, G, \quad R_i(u) = r_{i,u}\, G, \quad S_{i,u} = r_{i,u}\, \mathsf{pub}_u, \\
& s_i^{(j)}(u) \in [0,q), \quad s_i^{(j)}(u)\, G = \textstyle\sum_{k<t} u^k\, C_i^{(j)}(k), \quad h_E = \mathsf{H_P}(\mathbf{pub} \,\|\, \mathbf{E}_i), \\
& \sigma_i^{(j)}(u) = s_i^{(j)}(u) + \mathsf{H_P}(\mathsf{ctx}_{i,u} \,\|\, j \,\|\, S_{i,u}) \bmod q, \quad \mathbf{w} = \mathbf{C}_i \,\|\, \mathbf{pub} \,\|\, \mathbf{E}_i, \\
& h_C = \mathsf{H_P}\big(\mathtt{eid}, i, t, g_{i,0}, \ldots, g_{i,K_{\max}-1}\big); \\
\mathcal{R}_{\mathsf{fin}} :\ & t \le a \le n \le N_{\max}, \quad I_\ell \in [n], \quad h_{C,\ell} = \mathsf{H_P}\big(\mathtt{eid}, I_\ell, t, g_{\ell,0}, \ldots, g_{\ell,K_{\max}-1}\big), \\
& \overline{C}^{(j)}(k) = \textstyle\sum_{\ell<a} C_\ell^{(j)}(k), \quad P_j = \overline{C}^{(j)}(0), \quad D_{j,i} = \textstyle\sum_{k<t} i^k\, \overline{C}^{(j)}(k), \\
& \mathbf{w} = \mathbf{I} \,\|\, \mathbf{h}_C \,\|\, P_0 \,\|\, \mathbf{D}_0 \,\|\, \cdots \,\|\, P_{K_{\max}-1} \,\|\, \mathbf{D}_{K_{\max}-1}, \quad B_j = \mathsf{H_P}(1, j, P_j, \mathbf{D}_j), \\
& d = \mathsf{H_P}\big(2, \mathtt{eid}, t, n, a, K_{\max}, |\mathbf{w}|, \mathsf{H_P}(0, \mathbf{I} \,\|\, \mathbf{h}_C), B_0, \ldots, B_{K_{\max}-1}\big); \\
\mathcal{R}_{\mathsf{part}} :\ & (D_{j,i}, \delta_i) = e_{j,i}\,(G, C_1), \quad (A_i, B_i) = w_i\,(G, C_1), \\
& z_i\,(G, C_1) = (A_i, B_i) + c_i\,(D_{j,i}, \delta_i); \\
\mathcal{R}_{\mathsf{comb}} :\ & t \le s, \quad m, \mathsf{sk}_{\mathsf{org}} \in [0,q), \quad \mathsf{PK}_{\mathsf{org}} = \mathsf{sk}_{\mathsf{org}}\, G, \quad \textstyle\sum_{k\le s} \lambda_k\, x_k^r\, G = [r=0]\, G \quad (r < s), \\
& C_2 = m\, G + \textstyle\sum_{k\le s} \lambda_k\, \delta_{x_k} + \mathsf{sk}_{\mathsf{org}}\, C_1, \quad h = \mathsf{H_P}(\mathtt{eid}, \mathtt{aid}, \mathtt{ctIdx}, t, s, \mathbf{w}), \\
& \mathbf{w} = C_1 \,\|\, C_2 \,\|\, \mathsf{PK}_{\mathsf{org}} \,\|\, \{x_k\} \,\|\, \{\delta_{x_k}\} \in \mathbb{F}_p^{6+3N_{\max}}.
\end{aligned}$$

The range checks and the bounded-quotient mask reduction $\mathsf{H_P}(\cdot) = \mathsf{quot} \cdot q + \mathsf{mask}$ (Appendix A.3) make every share and mask canonical over the integers. In $\mathcal{R}_{\mathsf{fin}}$ the a rows name distinct accepted dealers I_ℓ and their stored digests $h_{C,\ell}$, which bind each dealer's $2K_{\max}N_{\max}$ commitment words as witness through the nested $g_{\ell,j}$; inactive rows, coefficients $k \geq t$ and positions $i > n$ are zero or the identity, and $\mathbf{w}$ has $2N_{\max} + K_{\max}(2 + 2N_{\max}) = 1\,120$ words. The contract matches the rows against its frozen QUAL, reads every P_j and $\mathbf{D}_j$ from the commitment-bound calldata and stores P_j and $\mathsf{root_j}$ (§4). $\mathcal{R}_{\mathsf{part}}$ keeps (A_i, B_i, z_i) a self-contained DLEQ proof [15]; $\mathcal{R}_{\mathsf{comb}}$ proves knowledge of the $\mathsf{sk_{org}}$ read from the application record (0 for an automatic application). Before the pairing check the contract also ties inputs to state (t, n, a, the rows and $h_{C,\ell}$, the digest at input 4, C_1, the Merkle path of $D_{j,i}$ against $\mathsf{root_j}$, $\mathsf{PK_{org}}$, the stored $\mathsf{keccak}(\delta_{x_k})$) and gates both decryption calls (§4).

Canonical interpolation. Since $\mathbb{F}_p \neq \mathbb{F}_q$ the λ_k cannot be computed by division; $\mathcal{R}_{\mathsf{comb}}$ checks the square Vandermonde system on points, where arithmetic is modulo q. For s distinct non-zero nodes, which the contract enforces, its unique solution is the canonical Lagrange vector at 0 of $\mathcal{Q}$. Correctness and uniqueness differ: the first t rows already make $\sum_k \lambda_k \delta_{x_k}$ interpolate the shared f whichever solution the prover picks, since a perturbation μ in their nullspace contributes $(\sum_k \mu_k f(x_k))\, C_1 = 0$; the remaining $s - t$ rows make the vector canonical, so a given set of partials admits exactly one accepted witness (§6).

Compact contribution transcript. The contribution's calldata carries no padding: $l = K_{\max}(2t + n) + 5n$ words, sized by the epoch's public (t, n), in the order commitments (key-major, t per key), recipient indexes, recipient keys, ephemerals, masked shares (key-major, n per key); the contract checks the $3n$ index and key words against the committee snapshot hashed at the last claim. The circuit's arrays have capacity $N_{\max}$, so it folds every word w under a gate $b \in \{0, 1\}$ computed from the public counts ($[k < t]$ for a coefficient, $[u \leq n]$ otherwise): $\mathsf{acc} \leftarrow \mathsf{acc} + b\,\pi\,w$, $\pi \leftarrow \pi\,(1 + b(\rho - 1))$, $\mathsf{cnt} \leftarrow \mathsf{cnt} + b$, then asserts $\mathsf{cnt} = l$ and $\mathsf{acc} = \beta$. An inactive entry neither contributes nor advances the exponent, so the fold equals BRLC_ρ of the active words in calldata order and Lemma 1 applies with l fixed by (t, n).

5.1 Binding Random Linear Combinations

Passed naively, $\mathcal{R}_{\mathsf{deal}}$ would have $2K_{\max}t + (3 + K_{\max})\,n$ public scalars, over 4 M gas of verifier scalar multiplications at $n = 20$, $t = 10$ [11]; instead each circuit exposes a digest d of its witness vector $\mathbf{w} \in \mathbb{F}_p^l$ and one word $\beta = \mathsf{BRLC}_\rho(\mathbf{w}) := \sum_{k=1}^{l} \rho^k w_k \bmod p$, which the contract recomputes over the l calldata words $\mathbf{v}$ (Freivalds [23]). It binds only if ρ is independent of *both* vectors: were ρ derived from the prover's digests alone, the prover would know it before choosing the calldata and could satisfy one linear equation over $\mathbb{F}_p$—change v_a by η and v_b by $-\eta\rho^{a-b}$—freeing the words the contract reads straight from calldata, P_j among them; a witness word no digest fixes fails dually. The challenge therefore covers both sides, $\rho := \mathsf{keccak}(\mathtt{eid} \,\|\, \mathsf{dom} \,\|\, \mathsf{keccak}(d \,\|\, \mathsf{keccak}(\mathbf{v}))) \bmod p$, where d

absorbs every witness word the circuit commits to, inactive slots masked identically throughout, and dom is a per-relation constant; the anchors are $h_C \,\|\, h_E$, d and $h \,\|\, m$ respectively.

Lemma 1 (BRLC binding). *Let* $\mathbf{v} \neq \mathbf{w} \in \mathbb{F}_p^l$ *be fixed and* $\rho = x \bmod p$ *for uniform* $x \in [0, 2^{256})$*; then* $\Pr[\mathsf{BRLC}_\rho(\mathbf{v}) = \mathsf{BRLC}_\rho(\mathbf{w})] \leq \gamma\, l/p$*,* $\gamma = \lceil 2^{256}/p \rceil\, p/2^{256} < 1.135$*. Hence, with* keccak *a random oracle [7], a* Q*-query prover that has the contract accept* l *canonical calldata words* $\mathbf{v}$ *and a proof with public inputs* $d = \mathsf{H_P}(\mathbf{w})$*,* ρ *as above and* $\mathsf{BRLC}_\rho(\mathbf{w})$*,* $\mathbf{w} \neq \mathbf{v}$*, succeeds with probability at most* $Q\gamma l/p$ *plus its advantage against* $\mathsf{H_P}$*'s collision resistance and the proof system's knowledge soundness.*

Proof. The polynomial $\sum_{k=1}^{l}(w_k - v_k)X^k$ is non-zero of degree at most l, so it has at most l roots [39], each hit with probability at most $\lceil 2^{256}/p \rceil / 2^{256} = \gamma/p$. The accepted ρ answers a query $\mathsf{keccak}(\mathtt{eid} \,\|\, \mathsf{dom} \,\|\, \mathsf{keccak}(d \,\|\, h))$ in which h fixes $\mathbf{v}$ and d fixes the extracted $\mathbf{w}$ unless a hash collides, so both are determined before the answer is drawn; a union bound over Q queries finishes. □

By construction both vectors have the same length l—the circuit's capacity or, for $\mathcal{R}_{\mathsf{deal}}$, the gate count the circuit asserts from the public (t, n) that also size the calldata—the contract rejects words $\geq p$, so a transcript has one encoding; only the root bound is statistical, below $2^{-242.7}$ at the largest transcript, $l = 1\,696$ ($t = n = 32$), against $2^{-242.9}$ for a uniform challenge.

6 Security analysis

Model. We consider one epoch of §4, conditional on its selected committee. A static adversary controls f members, arbitrary other organizers and submitters, and transaction scheduling within the chain assumptions; it sees the public transcript and may rush honest dealings. Honest nodes validate subgroup membership before releasing partials. Secrecy concerns an honestly encrypted challenge with fresh randomness, not plaintext provenance, ballot validity, or secrecy after authorized release.

We use the following assumptions. (a) DDH is hard in the prime-order group $\mathcal{G} = \langle G \rangle$. (b) $\mathsf{H_K}$ is a 256-bit random oracle; possession-proof challenges reduce its output modulo q, BRLC challenges modulo p. (c) $\mathsf{H_P}$ is collision resistant for transcript binding; its share-mask invocations are additionally idealized as a random oracle over $\mathbb{F}_p$ [7]; collision resistance alone does not imply mask secrecy. (d) Independently generated circuit reference strings provide auxiliary-input knowledge soundness and multi-theorem zero knowledge; Groth16's soundness analysis inherits its generic-group idealization [27]; compromised setup is excluded. (e) Honest parties use independent randomness and authenticated keys; pool-key secrecy requires $f < t$, $n - f \geq t$, and an honest dealer in QUAL—properties of the selected epoch, not guarantees of parameter floors. (f) Contribution proofs satisfy auxiliary-input, multi-theorem *weak simulation-extractability*: after polynomially many simulated proofs of adaptively chosen,

possibly false statements, witnesses are extractable from accepting adversarial statements never submitted to the simulator. We use Groth16's weak simulation-extractability in the algebraic group model under the hypotheses of [5]; other circuit setups and protocol oracles are simulated within the adversary wrapper, extraction may be white-box and algebraic, and online black-box extraction is not assumed.

Let ϵ_{ks} denote aggregate knowledge-soundness/extraction failure, and ϵ_{bind} the failure bound of Lemma 1, with all its encoding, digest-coverage, gating and context hypotheses and the actual modulo-p challenge distribution, plus remaining digest/Merkle collisions. Write $\epsilon_0 = \epsilon_{\mathrm{ks}} + \epsilon_{\mathrm{bind}} + \epsilon_{\mathrm{rng}}$ with $\epsilon_{\mathrm{rng}} \le N_{\mathrm{sc}}/q$, the cost of idealizing N_{sc} prescribed nonzero scalar samples as uniform over $\mathbb{F}_q$.

Lemma 2 (Dealing correctness). *Except with probability $\epsilon_{\mathrm{ks}} + \epsilon_{\mathrm{bind}}$, every accepted contribution encrypts evaluations of degree-below-t polynomials over $\mathbb{F}_q$, consistent with its coefficient commitments; its stored digest computationally binds all $K_{\max}$ commitment vectors.*

Proof. Soundness and Lemma 1 bind a relation witness to canonical calldata; coefficient commitments and share equations identify the polynomial evaluations, and Appendix A.3 establishes canonical masking arithmetic. Both parties compute the same ECDH point, so recipient subtraction recovers those evaluations.

Consequently, finalization binds $P_j = A_j(0)G$ and every committee member's $D_{j,i} = A_j(i)G$ under $\mathsf{root_j}$, where $A_j = \sum_{d \in \mathsf{QUAL}} f_d^{(j)}$, and root-checked partials satisfy $\delta_i = A_j(i)C_1$; the combine relation's Vandermonde check (§5) pins the canonical interpolation vector, and with the organizer-secret relation this proves the recovered message point correct.

Secrecy experiment. The adversary may register applications, submit arbitrary ciphertexts, and obtain every policy-admitted reveal and honest partial on other applications, including for copied challenge components. For pool-key secrecy, the target has never released an honest partial and the experiment ends before its gate first opens. The challenge encrypts one of two chosen messages; advantage is $|\Pr[\hat{b} = b] - 1/2|$, and inadmissible or aborted executions return a random guess, so bounds are not renormalized by successful epoch completion. There is no target-key decryption oracle and no same-application CCA claim.

Theorem 1 (Automatic-application secrecy and isolation, conditional). *Under assumptions (a)–(f), the experiment for an automatic application under its uniquely claimed pool key satisfies* $\mathrm{Adv}_{\mathrm{auto}}(\mathcal{A}) \le L\,\mathrm{Adv}^{\mathrm{ddh}}_{\mathcal{G}}(\mathcal{B}) + R\epsilon_{\mathrm{HE}} + \epsilon_{\mathrm{zk}} + \epsilon_{\mathrm{se}} + \epsilon_0$. *Here $L \le K_{\max}(n-f)$ accounts for guessing the target slot and an honest accepted dealer, $R \le (n-f)^2$ counts honest-dealer/honest-recipient rows, and ϵ_{HE} is the per-row hashed-encryption hybrid loss; ϵ_{zk} and ϵ_{se} aggregate, respectively, simulation loss and weak-simulation-extractability failure over all relevant contributions and hybrids, and reduction running times include extraction overhead.*

Proof. Proof sketch. In $H_0 \to H_1$, replace honest contribution proofs on true statements by simulated proofs. In $H_1 \to H_2$, replace each honest-recipient row's $K_{\max}$ masked shares by independent uniform placeholders, recomputing its digest and simulated proof; one ECDH secret serves the row, but the key index separates its hash inputs, and the transition includes encryption security and reduction bias, not merely zero knowledge. The modified honest dealing statements may be false, but every accepted corrupt contribution has a statement never simulated, so assumption (f), together with Lemma 1, preserves corrupt-dealing correctness; during the encryption-row reductions, extraction after finalization supplies the corrupt polynomials even when the embedded recipient's secret is unknown (Appendix A.2). In H_2, embed $X = uG$ in one honest dealer's target-key constant commitment. Interpolation in the exponent supplies coefficient commitments while fixing the $f < t$ corrupt recipients' evaluations to known scalars; honest-recipient target evaluations need not be known because their ciphertexts are placeholders, and finalization still has a genuine point-valued witness. In the final reduction all recipient encryption secrets are known: decrypt each adversarial dealing at t honest positions (using $n - f \geq t$) and interpolate its polynomial. Including other honest dealers gives a known, possibly adaptively chosen offset a with $P_j = X + aG$, and every other key's shares are known, so its oracles remain answerable. For a DDH tuple (G, X, Y, Z), use $(C_1^\star, C_2^\star) = (Y, m_b G + aY + Z)$: the DH case is the challenge encryption, and independent uniform Z hides b. Appendix A.2 supplies the bookkeeping. This proves computational isolation, not uniformity of adversarially biased output keys [24].

Theorem 2 (Locked-application secrecy). *Under (a)–(d) and honest randomness, a locked target with an independent, unrevealed organizer secret remains secret even after disclosure of every pool share, including against $f \geq t$ members without that secret:* $\mathrm{Adv}_{\mathrm{locked}}(\mathcal{A}) \leq Q_A\big(\mathrm{Adv}_{\mathcal{G}}^{\mathrm{ddh}}(\mathcal{B}) + Q_K/q\big) + \epsilon_0$*, where Q_A bounds honest organizer registrations and Q_K counts $\mathsf{H_K}$ queries. Conversely, under (a)–(f), giving the adversary the target's correctly generated organizer secret preserves Theorem 1's bound while the target gate remains closed.*

Proof. Proof sketch. Recover the pool polynomials from t honest recipients; if $n - f < t$, use (d)'s contribution extractor before any organizer-proof simulation. Embed $X = uG$ as the organizer key, simulate its possession proof using (b), and, knowing the pool scalar x_j, form $(Y, m_b G + x_j Y + Z)$; independent uniform Z hides the message even given all committee shares. No SNARK simulation or pool independence is needed for this direction. For the organizer-adversary direction, adding its known term $\mathsf{sk_{org}} C_1$ transports the automatic experiment. Appendix A.1 details both interfaces.

What the gate does and does not promise. The reveal makes all admitted ciphertexts eligible under one application-wide gate, subject to the decryption window, and with fewer than t corrupt members honest nodes release no partial before it opens. Nevertheless, t colluding members defeat an automatic application's

Table 3. Circuit size and proving time at $N_{\max} = 32$, $K_{\max} = 16$ (mean of five proofs, 32 threads, witness solving included, key loading excluded). Inputs: public inputs of the verifier. Transcript: BRLC-committed calldata words, for the unpadded contribution at $(n, t) = (4, 3)$ and $(32, 22)$; a partial carries its 15 inputs and a 5-word Merkle path instead.

Circuit	Constraints	Inputs	Transcript	Proving time
Contribution	5 904 167	8	180 – 1 376	3 540 ms
Finalization	2 328 130	7	1 120	2 259 ms
Partial decryption	29 026	15	—	32 ms
Combine	287 338	9	102	213 ms

embargo, an organizer with t members decrypts locked ciphertexts off chain without revealing, a closing deadline cannot revoke published partials, and an organizer-secret leak permits an unauthorized reveal but does not bypass a still-closed window. Eligibility is neither simultaneous delivery nor fairness, and an authorized submitter still chooses which ciphertexts or aggregates are admitted. Decryption needs t available share holders and an open gate.

7 Implementation and Evaluation

The implementation consists of the node and the four circuits in Go on gnark v0.16.3 [9], the three contracts in Solidity 0.8.28 (via-IR, optimizer runs 1, Cancun rules) and a TypeScript client. Every figure below is for the fixed capacities $N_{\max} = 32$, $K_{\max} = 16$; $K_{\max}$ is a release constant of circuit, verifier and contract, not a caller's choice. The Groth16 artifacts come from a single-party setup pinned by hash in the node; a production deployment must replace them by a multi-party ceremony, which changes neither constraint counts nor gas.

Methodology. Constraint counts are those of the compiled R1CS; proving times are wall-clock per proof, mean of five, over the 32 logical threads of an idle AMD Ryzen 9 9950X3D (64 GiB), witness solving included, key loading excluded; peak prover memory (maximum resident set, key and circuit loaded) is 9.3 GB for the contribution and 5.5 GB for the finalization. Gas is `gasUsed` from the receipts of a local Anvil chain (Cancun rules) running the real verifiers, pairing check, calldata and intrinsic cost included, at $(n, t) = (4, 3)$, $(32, 22)$ and over the sweep of Table 4. Latency and behaviour under delayed or offline members were not measured.

Results. In Table 3 the contribution circuit scales with $K_{\max}N_{\max}t$ through its $K_{\max}n$ Horner evaluations and $K_{\max}N_{\max}$ masked shares: doubling $K_{\max}$ from 8 to 16 took it from 3.06 M constraints, 1.57 s and 5.0 GB to 5.90 M, 3.54 s and 9.3 GB. The finalization re-absorbs every accepted dealer's $2K_{\max}N_{\max}$ commitment words and evaluates $K_{\max}N_{\max}$ share commitments by Horner; the partial is independent of $N_{\max}$ and the combine dominated by its Vandermonde

Table 4. Transaction gas (10^3) against the real verifiers on a local development chain (receipts' `gasUsed`, $N_{\max} = 32$, $K_{\max} = 16$). Slope: gas per additional member over $n = 4, 8, \ldots, 32$, $t = \lceil 2n/3 \rceil$ (per additional partial for the combine); the other calls stay within 1% except claimSlot, whose average falls as the 205k seed-resolving claim is spread over more members. Registration: locked (including a 168k subgroup check of the organizer key) / automatic. The createEpoch resolving the first seed costs 150k.

Call	n=4, t=3	n=32, t=22	Slope	Paid by
createEpoch	133	133	—	creator / epoch
claimSlot	137	121	avg.	member
submitContribution	500	1 314	+29.1	member
finalizeEpoch	2 202	2 811	+21.7	member / epoch
registerApplication	619 / 232	621 / 232	—	organizer / application
revealOrganizerSecret	226	224	—	organizer / locked app.
submitCiphertext	103	103	—	submitter / ciphertext
submitPartialDecryption	402	402	—	t members / ciphertext
combineDecryption	411	484	+3.9	member / ciphertext

check. A node thus spends about 3.5 s of wall-clock time on 32 threads per epoch on its contribution, 2.3 s more if it is the one that finalizes, 32 ms per partial and 0.21 s per combine, which recovers m from $m\,G$ by baby-step giant-step [41], capped at 2^{50} in the node (a 256 MiB table) and 2^{32} in the browser; a worst-case plaintext below 2^{50} takes 3.7 s on 32 threads. In Table 4 every proof-carrying call pays about 250k gas for the Groth16 verification itself [11]; the rest is calldata and storage. The contribution's transcript grows with t and n, from 180 words to 1 376 (43 KiB), hence its slope. The finalization's 1 120 words (35 KiB) are fixed by $N_{\max}$ and $K_{\max}$, absent members' slots being identity points of mostly zero bytes at 4 gas each; the call also checks one storage row per accepted dealer, hashes $K_{\max}$ Merkle leaves per member and writes 48 cold storage slots (1.06 M gas by the fee schedule). The fixed verifier does not make a constant-cost transaction: a contribution rises from 500k to 1 314k gas across the sweep.

End-to-end cost. Adding the rows of Table 4, an epoch that yields all $K_{\max} = 16$ keys—creation, n claims, n contributions, one finalization—costs 4.89 M gas at $n = 4$ and 48.9 M at $n = 32$, i.e. 0.31 M and 3.06 M per key, the contributions being 41% and 86% of it; every key is paid for at finalization, claimed or not. Decrypting one ciphertext (submission, t partials, one combine) costs 1.72 M at $t = 3$ and 9.44 M at $t = 22$; these totals exclude application registration, the 0.22 M reveal of a locked application, operator registration and deployment. Two baselines on the same circuits, toolchain and machine place these numbers. A single-key build ($K_{\max} = 1$) has a 587 022-constraint contribution circuit proving in 0.45 s and a contribution transaction of 468k gas at $n = 4$ and 535k at $n = 32$, so dealing 16 keys costs 1.07–2.5× dealing one, and its epoch, yielding one usable key, costs 3.1 M gas at $n = 4$, ten times the pooled per-key figure. The previous

design of this system, $K_{\max} = 8$ with a proof-less finalization and one 187 495-constraint activation proof and 508–809k transaction per key, cost 0.95 M per key at $n = 4$ and 5.09 M at $n = 32$ with the pool fully used; one finalization proof and unpadded calldata save 68% and 40% per key, for a 2.25× longer contribution proof and a 2.3 s finalization proof on one node.

Artifact. The implementation, circuits, contracts, test vectors, benchmark procedure and the regression tests for the transcript binding of §5.1 and the interpolation check are available as anonymized supplementary material, which distinguishes the experimental single-party setup from a production ceremony.

8 Limitations and Conclusion

Committee, epochs and pool. The seed's proposer can bias the lottery, slots go to the fastest claimants, and a rushing dealer biases pool keys by choosing which of its dealings enter QUAL (§6). Finalization has no deadline and is one 2.3 M-constraint proof (2.3 s, 5.5 GB) that some node must produce before any key of the epoch exists—an epoch-wide liveness dependency on one prover, served in seed-derived stagger slots, where the previous design went live without a proof; a member's contribution prover peaks at 9.3 GB for 3.5 s per epoch, and $K_{\max}$ is a release constant that doubled both when doubled. Re-registration protects only future encryptions. An epoch serves at most $K_{\max}$ applications, further ones waiting for the next; $K_{\max} - 1$ permissionless registrations let createEpoch run early at the committee's expense, which also pays t partials per admitted ciphertext, with no fee or admission list to deter either.

Organizer key, submissions and trust base. $\mathsf{sk}_{\mathtt{org}}$ is a single point of failure: losing it makes a locked application undecryptable, reusing it lets one reveal open every application it backs, and leaking it permits an unauthorized but visible reveal, or a quiet decryption if t members also collude (§6); published partials are irrevocable. Admitted submitters can maul same-application ciphertexts, so admission is the application's responsibility; identifiers being first come, first served, clients must verify registrant and key on chain. An out-of-range plaintext costs each combiner a failed discrete-logarithm search; tainting the offending application–submitter pair bounds this per submitter, not against fresh addresses. The single-party setup's toxic waste permits false plaintexts (combine) or a chosen P_j (dealing), the proof library is part of the trusted base, and nothing here is post-quantum; future work: admission economics, beacon-based selection [35,19], proactive refresh [30].

Conclusion. Proof-carrying dealings give an EVM contract a publicly verifiable DKG with one fixed-size verification per contribution and no dispute phase; keys dealt as a pool and finalized by one proof isolate the applications one committee serves, and an organizer key adds a one-shot, application-wide release.

References

1. Abdalla, M., Bellare, M., Rogaway, P.: Dhies: An encryption scheme based on the diffie–hellman problem. Tech. rep., Cryptology ePrint / Full version on the authors' webpages (Sep 2001), https://web.cs.ucdavis.edu/~rogaway/papers/dhies.pdf, full version dated September 18, 2001
2. Abraham, I., Bacho, R., Loss, J., Stern, G.: Nearly quadratic asynchronous distributed key generation. IACR ePrint 2025/006 (2025)
3. Abraham, I., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G.: Bingo: Adaptivity and asynchrony in verifiable secret sharing and distributed key generation. In: Advances in Cryptology – CRYPTO 2023. pp. 39–70. Springer (2023)
4. Bacho, R., Kavousi, A.: SoK: Dlog-based distributed key generation. In: IEEE Symposium on Security and Privacy (S&P). pp. 614–632 (2025)
5. Baghery, K., Kohlweiss, M., Siim, J., Volkhov, M.: Another look at extraction and randomization of Groth's zk-SNARK. In: Financial Cryptography and Data Security, FC 2021. LNCS, vol. 12674, pp. 457–475. Springer (2021). https://doi.org/10.1007/978-3-662-64322-8_22
6. Bebel, J., Ojha, D.: Ferveo: Threshold decryption for mempool privacy in BFT networks. Cryptology ePrint Archive, Paper 2022/898 (2022)
7. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: 1st ACM Conference on Computer and Communications Security (CCS). pp. 62–73 (1993)
8. Benhamouda, F., Halevi, S., Krawczyk, H., Ma, Y., Rabin, T.: SPRINT: High-throughput robust distributed Schnorr signatures. In: Advances in Cryptology – EUROCRYPT 2024. LNCS, Springer (2024), iACR ePrint 2023/427
9. Botrel, G., Piellard, T., Housni, Y.E., Kubjas, I., Tabaie, A.: ConsenSys/gnark: a fast zk-SNARK library that offers a high-level API to design circuits. Software, https://github.com/Consensys/gnark (2023). https://doi.org/10.5281/zenodo.5819104
10. Canetti, R., Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Adaptive security for threshold cryptosystems. In: Advances in Cryptology – CRYPTO '99. LNCS, vol. 1666, pp. 98–115. Springer (1999)
11. Cardozo, A.S., Williamson, Z.: EIP-1108: Reduce alt_bn128 precompile gas costs. Ethereum Improvement Proposals (2018), https://eips.ethereum.org/EIPS/eip-1108
12. Cascudo, I., David, B.: SCRAPE: Scalable randomness attested by public entities. In: Applied Cryptography and Network Security – ACNS 2017. LNCS, vol. 10355, pp. 537–556. Springer (2017)
13. Cascudo, I., David, B.: ALBATROSS: Publicly attestable batched randomness based on secret sharing. In: Advances in Cryptology – ASIACRYPT 2020. LNCS, vol. 12493, pp. 311–341. Springer (2020)
14. Cash, D., Kiltz, E., Shoup, V.: The twin diffie–hellman problem and applications. In: Advances in Cryptology – EUROCRYPT 2008. Lecture Notes in Computer Science, vol. 4965, pp. 127–145. Springer (2008). https://doi.org/10.1007/978-3-540-78967-3_8
15. Chaum, D., Pedersen, T.P.: Wallet databases with observers. In: Advances in Cryptology – CRYPTO '92. LNCS, vol. 740, pp. 89–105. Springer (1993)
16. Das, S., Xiang, Z., Kokoris-Kogias, L., Ren, L.: Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In: 32nd USENIX Security Symposium. pp. 5359–5376 (2023)

17. Das, S., Yurek, T., Xiang, Z., Miller, A., Kober, L., Gao, L.: Practical asynchronous distributed key generation. In: 2022 IEEE Symposium on Security and Privacy (SP). pp. 2518–2534. IEEE (2022)
18. Desmedt, Y., Frankel, Y.: Threshold cryptosystems. In: Advances in Cryptology – CRYPTO '89. LNCS, vol. 435, pp. 307–315. Springer (1990)
19. drand contributors: drand: A distributed randomness beacon daemon. Project website (2024), `https://drand.love/`, accessed: 2026-04-29
20. Feldman, P.: A practical scheme for non-interactive verifiable secret sharing. In: 28th Annual Symposium on Foundations of Computer Science. pp. 427–438. IEEE (1987)
21. Feng, H., Tang, Q.: Asymptotically optimal adaptive asynchronous common coin and DKG with silent setup. In: Advances in Cryptology – CRYPTO 2025. pp. 647–680. Springer (2025)
22. Fouque, P.A., Stern, J.: One round threshold discrete-log key generation without private channels. In: Public Key Cryptography – PKC 2001. LNCS, vol. 1992, pp. 300–316. Springer (2001)
23. Freivalds, R.: Probabilistic machines can use less running time. In: Information Processing 77, Proceedings of IFIP Congress 77. pp. 839–842. North-Holland (1977)
24. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. In: Advances in Cryptology – EUROCRYPT '99. LNCS, vol. 1592, pp. 295–310. Springer (1999), full version: Journal of Cryptology 20(1):51–83, 2007
25. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure applications of Pedersen's distributed key generation protocol. In: Topics in Cryptology – CT-RSA 2003. LNCS, vol. 2612, pp. 373–390. Springer (2003)
26. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: 30th USENIX Security Symposium. pp. 519–535 (2021)
27. Groth, J.: On the size of pairing-based non-interactive arguments. In: Advances in Cryptology – EUROCRYPT 2016. Lecture Notes in Computer Science, vol. 9666, pp. 305–326. Springer (2016). `https://doi.org/10.1007/978-3-662-49896-5_11`
28. Groth, J.: Non-interactive distributed key generation and key resharing. IACR Cryptology ePrint Archive, Report 2021/339 (2021), `https://eprint.iacr.org/2021/339`
29. Gurkan, K., Jovanovic, P., Maller, M., Meiklejohn, S., Stern, G., Tomescu, A.: Aggregatable distributed key generation. In: Advances in Cryptology – EUROCRYPT 2021. LNCS, vol. 12696, pp. 147–176. Springer (2021)
30. Herzberg, A., Jarecki, S., Krawczyk, H., Yung, M.: Proactive secret sharing or: How to cope with perpetual leakage. In: Advances in Cryptology — CRYPTO '95. Lecture Notes in Computer Science, vol. 963, pp. 339–352. Springer (1995)
31. Hisil, H., Wong, K.K.H., Carter, G., Dawson, E.: Twisted Edwards curves revisited. In: Advances in Cryptology – ASIACRYPT 2008. LNCS, vol. 5350, pp. 326–343. Springer (2008)
32. Kate, A., Goldberg, I.: Distributed key generation for the internet. In: 29th IEEE International Conference on Distributed Computing Systems (ICDCS). pp. 119–128 (2009)
33. Kate, A., Mangipudi, E.V., Mukherjee, P., Saleem, H., Thyagarajan, S.A.K.: Non-interactive VSS using class groups and application to DKG. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS 2024). ACM (2024), cryptology ePrint Archive, Paper 2023/451

34. Kokoris-Kogias, E., Malkhi, D., Spiegelman, A.: Asynchronous distributed key generation for computationally-secure randomness, consensus, and threshold signatures. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. pp. 1751–1767 (2020)
35. Micali, S., Rabin, M.O., Vadhan, S.P.: Verifiable random functions. In: 40th Annual Symposium on Foundations of Computer Science (FOCS 1999). pp. 120–130. IEEE Computer Society (1999)
36. Pedersen, T.P.: A threshold cryptosystem without a trusted party. In: Advances in Cryptology—EUROCRYPT'91. pp. 522–526. Springer (1991)
37. Schindler, P., Judmayer, A., Stifter, N., Weippl, E.: ETHDKG: Distributed key generation with Ethereum smart contracts. Cryptology ePrint Archive, Report 2019/985 (2019)
38. Schoenmakers, B.: A simple publicly verifiable secret sharing scheme and its application to electronic voting. In: Advances in Cryptology – CRYPTO '99. LNCS, vol. 1666, pp. 148–164. Springer (1999)
39. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. Journal of the ACM **27**(4), 701–717 (1980)
40. Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (1979). `https://doi.org/10.1145/359168.359176`
41. Shanks, D.: Class number, a theory of factorization, and genera. In: Proceedings of Symposia in Pure Mathematics. vol. 20, pp. 415–440. AMS (1971)
42. Shoup, V., Gennaro, R.: Securing threshold cryptosystems against chosen ciphertext attack. Journal of Cryptology **15**(2), 75–96 (2002), preliminary version in EUROCRYPT '98
43. Sober, M., Kobelt, M., Scaffino, G., Kaaser, D., Schulte, S.: Distributed key generation with smart contracts using zk-SNARKs. In: Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing (SAC). pp. 229–238. ACM (2023). `https://doi.org/10.1145/3555776.3577677`, best Paper Award, Distributed Systems Track
44. Stadler, M.: Publicly verifiable secret sharing. In: Advances in Cryptology – EUROCRYPT '96. LNCS, vol. 1070, pp. 190–199. Springer (1996)
45. Team, S.N.: Shutter: Distributed key generation with applications to front-running protection. Cryptology ePrint Archive, Report 2024/1981 (2024), deployed on Gnosis Chain since October 2022
46. Wuille, P.: BIP-0032: Hierarchical deterministic wallets. Bitcoin Improvement Proposal (2012), `https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki`, accessed: 2026-04-29
47. Zhang, H., Duan, S., et al.: Practical asynchronous distributed key generation: Improved efficiency, weaker assumption, and standard model. In: IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). pp. 568–581 (2023)

A Reduction details

A.1 Locked-application bookkeeping

First finish the epoch using real circuit proofs. When $n - f \geq t$, Lemma 2 lets the reduction recover each adversarial polynomial from t honest recipients; otherwise it invokes assumption (d)'s knowledge extractor on the accepted contributions,

adding extractor running time and failure probability ϵ_{ks}—not a straight-line reconstruction claim. Extraction precedes organizer registration, so it needs no soundness after simulated SNARK proofs.

To simulate a possession proof for X, sample $h \leftarrow \{0,1\}^{256}$ and $z \leftarrow \mathbb{F}_q$, and set $c = h \bmod q$, $T = zG - cX$, $\mathsf{H_K}(\mathsf{ctx}_{\mathrm{org}}, X, T) := h$, where the context holds the registration domain, epoch, and application identifier. Programming the *full* output preserves the actual reduced-challenge distribution; as T is uniform, an existing-query conflict costs at most Q_K/q per registration, so switching at most Q_A honest registrations costs $Q_A Q_K/q$.

Guess the challenge registration and embed the DDH element there; other organizer secrets remain known and their reveals are answered normally, while the target secret is neither revealed nor supplied through an organizer-DH oracle, although all committee partials could be disclosed. The challenge uses the known pool scalar as in Theorem 2; unsuccessful guesses return random bits, yielding the factor Q_A.

For the organizer-adversary direction, the experiment supplies the target's correctly generated organizer secret as auxiliary input; ciphertext translation adds its known multiple of C_1, and all target honest partials remain gated. This corollary does not analyze arbitrary malicious target-key generation merely from a successful possession verification.

A.2 Automatic-application hybrids and composition

For dealer d, recipient u, and key j, the actual mask structure is $S_{d,u} = r_{d,u}\mathsf{pub}_u$, $M_{d,u,j} = \mathsf{H_P}(\mathsf{dom}_{\mathrm{mask}}, \mathtt{eid}, 2^{16}d + u, j)$, and $\sigma_{d,u,j} = f_d^{(j)}(u) + \mathsf{H_P}(M_{d,u,j}, S_{d,u}.x, S_{d,u}.y) \pmod q$. Index bounds make the packing injective, and, except for hash collisions already charged to ϵ_{bind}, different keys have different outer-mask inputs despite sharing $S_{d,u}$.

After simulating honest contribution proofs, a per-row hashed-ElGamal argument [1] first replaces the row's DH secret by an independent random group point; unless that point is queried, its $K_{\max}$ hash outputs are independent field elements, so knowledge of other keys' shares and masks does not expose the target mask. Outside the aggregate extraction and collision bad events, a sufficient per-row bound is $\epsilon_{\mathrm{HE}} \le \mathrm{Adv}^{\mathrm{ddh}}_{\mathcal{G}}(\mathcal{B}_{\mathrm{row}}) + (Q_P + Q_K)/q + K_{\max}\beta$ with $\beta = \eta(q-\eta)/(pq)$ and $\eta = p - 7q$, where Q_P counts relevant outside queries to the mask oracle, Q_K/q covers simulation of the embedded recipient's registration proof, and β is exactly the statistical distance between $U(\mathbb{F}_p) \bmod q$ and $U(\mathbb{F}_q)$: η residues have eight preimages and the others seven.

The row reduction may not know its embedded recipient's secret. It nevertheless simulates the contribution phase without decrypting corrupt dealings, freezes QUAL, and extracts witnesses for the fixed accepted corrupt statements before answering service-phase queries; their polynomials determine honest virtual shares and other-key partials, outside the binding bad event. Only honest contributor contexts receive simulated contribution proofs, and static corruption, authenticated contributor indices and the one-contribution rule make every

accepted corrupt statement fresh for weak simulation-extractability, including throughout the row hybrids. White-box extraction suffices; internal rewinding must preserve the accepted statements and resume the saved main execution and oracle state, and all extraction failure and any truncation loss are included in ϵ_{se}. Honest encrypted rows have no decryption oracle, and virtual states retain the original mathematical shares rather than decrypting placeholders.

For the final DDH embedding, choose $t-1$ nonzero interpolation nodes containing all corrupt positions, assign them independent known evaluations v_z, let L_0, L_z be the basis polynomials for those nodes together with zero, and set $F(T)G = L_0(T)X + \sum_z L_z(T)v_zG$, a uniformly distributed degree-below-t polynomial in the ideal sampling experiment with known corrupt evaluations; every other honest polynomial is sampled normally. All recipient encryption secrets are now known, so t honest decryptions recover each adversarial polynomial without extraction. Finalization is genuinely provable from commitment points and their digests: its transcript digest covers the masked $2N_{\max} + K_{\max}(2 + 2N_{\max})$ words together with $\mathtt{eid}, t, n, |\mathsf{QUAL}|$, occupies input index 4 of 7, and enters the BRLC anchor, and no hidden polynomial scalar is its witness.

Guess the target slot and the lowest-index honest dealer in QUAL, testing both guesses before using the adversary's output; this gives $L \le K_{\max}(n-f)$ without assuming a dealer is guaranteed acceptance. The encryption hybrids run over all R rows before this guessing reduction, so their losses are additive outside L. Adaptive offsets may bias the final keys, but the recovered scalar offset still permits the final DDH translation. All reductions include their proving and extraction overhead.

A.3 Canonical share-mask reduction

Write $p = 7q + \eta$, where $0 < \eta < q$. For a canonical raw hash value $h \in [0, p)$, the circuit enforces $h = \nu q + \mu \pmod p$ with $0 \le \nu \le 7$, $0 \le \mu < q$, and $\nu = 7 \Longrightarrow \mu < \eta$. These bounds imply $\nu q + \mu < p$ over the integers, so the field equality is an integer equality, giving the unique canonical remainder $h \bmod q$; a remainder bound alone would not suffice. Masked-share addition also range-checks the share, mask, and result below q, and restricts its carry to $\{0, 1\}$; both sides of that equality are below $2q < p$, excluding native-field wraparound there as well.